Tutorial III.IV - Input and Output

Applied Optimization with Julia



Introduction

Welcome to this interactive Julia tutorial on working with external files! File Input/Output (I/O) operations are crucial in programming and data analysis, allowing us to persist data, share information between programs, and work with large datasets that don't fit in memory. In this tutorial, we'll cover reading and writing text files, handling CSV files, and working with delimited files using various Julia packages. These skills are fundamental for data preprocessing, analysis, and result storage in real-world applications.

Follow the instructions, write your code in the designated code blocks, and validate your results with `@assert` statements.

Section 1 - Working with Delimited Files

Delimited files, such as CSV (Comma-Separated Values), are a common way to store structured data. Each value in the file is separated by a specific character, often a comma. Julia's DelimitedFiles package makes it easy to work with these files.

> **Tip**
> The DelimitedFiles package ships with the course environment, so you can use it right away in this course. In your own projects, you first have to add it with `Pkg.add("DelimitedFiles")`, just like any other package.

```
using DelimitedFiles
```

Now, let's create a simple matrix and save it as a CSV file:

```
# Create a 3x3 matrix
new_data = [10 12 6; 13 25 1; 40 30 7]

# Create a new folder called "ExampleData"
# We use mkpath() instead of mkdir(), as mkpath() does nothing
# if the folder already exists - so you can safely re-run this block
mkpath("$(@__DIR__)/ExampleData")

# Write the matrix to a CSV file
open("$(@__DIR__)/ExampleData/matrix.csv", "w") do io
    writedlm(io, new_data, ',')
end

println("CSV file 'matrix.csv' written successfully to folder ExampleData!")
```

> **Note**
> Note that we used the `@__DIR__` macro to build the path. When you run a file as a script, `@__DIR__` returns the folder of that file, no matter where you start Julia from. In the Zed REPL, a cell does not belong to a file, so `@__DIR__` returns the folder Julia runs in instead, the same as `pwd()`. Run `pwd()` in a cell to see where your files end up.

Exercise 1.1 - Read a CSV File

Now it's your turn! Let's read the CSV file we just created. Make sure you have run the code block from Section 1 first, as it creates the file and defines `new_data`.

> **Tip**
> To learn how to use a Julia function, you can type `?` followed by the function name in the REPL (Julia's command-line interface). For example, `?readdlm` will show you information about the `readdlm()` function.

Use the `readdlm()` function to read the 'matrix.csv' file we just created. Save the result in a variable called `read_matrix`.

> **Important**
> We wrote the file with `','` as the delimiter, but `readdlm()` assumes whitespace by default. You have to tell `readdlm()` the delimiter we used when writing, e.g. `readdlm(path, ',')`.

```
# YOUR CODE BELOW
# Don't forget to use the @__DIR__ macro to get the correct file path!


# Test your answer
@assert read_matrix == new_data "The read matrix does not match the original
one.
    Did you pass the delimiter ',' to readdlm()?"
println("File 'matrix.csv' read successfully!")
```

Section 2 - Working with CSV Files and DataFrames

The CSV package in Julia provides powerful tools for reading and writing CSV files to and from DataFrames, a common requirement in data analysis and data science projects. This requires the CSV and DataFrames packages. Both ship with the course environment, so you can use them right away. In your own projects, you add CSV first, as you learned in the package tutorial:

```
import Pkg
Pkg.add("CSV")
```

Exercise 2.1 - Write a DataFrame to a CSV File

Write the following given DataFrame to a CSV file `table_out.csv` in the folder `ExampleData`. This can be done by using the function `CSV.write()`. To learn the syntax, ask the built-in help with `?` and the function name.

```
using CSV, DataFrames
data = DataFrame(Name = ["Elio", "Bob", "Yola"], Age = [18, 25, 29])
csv_file_path = "$(@__DIR__)/ExampleData/table_out.csv"
# YOUR CODE BELOW
```

```
# Test your answer
@assert isfile("$(@__DIR__)/ExampleData/table_out.csv") "Sorry, the file could
not be found.
    Have you followed all steps?"
println("CSV file 'table_out.csv' written successfully!")
```

Exercise 2.2 - Read in a CSV File

Read the CSV file `table_out.csv` in the folder `ExampleData` into the variable `read_data`. Here you can use the function `CSV.read()`, e.g.:

```
read_data = CSV.read("Path/datatable.csv", DataFrame)
```

> **Note**
> Note, that you need to provide a sink for the data when using `CSV.read()`, e.g. a DataFrame.

```
# YOUR CODE BELOW
# Again, don't forget to use the @__DIR__ macro to get the correct file path!
```

```
# Test your answer
@assert read_data[1,1] == "Elio"
println("CSV file 'table_out.csv' read successfully!")
```

Exercise 2.3 - Read the Brewery's Order File

Remember the brewery game from the lecture? Its order file now lands on your desk. The sales team exported it from a German spreadsheet: columns are separated by `;`, and numbers use a decimal comma, like `60,0`. The following code writes the file to `ExampleData/orders.csv` exactly as you received it:

```
mkpath("$(@__DIR__)/ExampleData")
write("$(@__DIR__)/ExampleData/orders.csv", """
week;crates;note
```

```
1;53;
2;60,0;
3;61;
4;;
5;56;
6;126;harbour festival
7;60;
7;60;
8;59;
""")
println("File 'orders.csv' written to the folder ExampleData!")
```

First, read the file with plain `CSV.read(path, DataFrame)` and print it. What type does the `crates` column have, and why? The printed table shortens long type names, so check with `eltype()`. Notice that CSV.jl found the `;` on its own: it can guess which character separates the columns, but not that `60,0` uses a decimal comma. Then read the file again into the variable `orders`, this time telling `CSV.read()` how the file was written: `delim=';'` and `decimal=','`.

> **Tip**
> Keyword arguments come after a semicolon: `CSV.read(path, DataFrame; delim=';', decimal=',')`. Other tools treat the comma as a thousands separator and read `60,0` as 600: that is what happened in the game.

```
# YOUR CODE BELOW
```

```
# Test your answer
@assert orders.crates[2] == 60.0 "Week 2 should read 60.0 crates. Did you set decimal=','?"
@assert ismissing(orders.crates[4]) "Week 4 has no number in the file, so it should be missing."
println("Order file read successfully!")
```

Exercise 2.4 - Clean the Order File

Two problems remain: week 7 appears twice, and week 4 is `missing`. Remove the duplicated row with `unique!()`, then replace the missing value with the median of the other weeks. Week 6 looks unusual too, but read its `note` before you touch it.

> **Tip**
> `median()` comes with the `Statistics` package (`using Statistics`; it ships with Julia). `skipmissing()` ignores missing values, and `coalesce.(column, value)` replaces every missing entry with `value`.

```
# YOUR CODE BELOW
```

```
# Test your answer
@assert nrow(orders) == 8 "Week 7 should appear only once. Did you remove the duplicated row?"
@assert count(ismissing, orders.crates) == 0 "There are still missing values in the crates column."
@assert orders.crates[4] == 60.0 "Week 4 should get the median of the other weeks, 60 crates."
@assert orders.crates[6] == 126 "The harbour festival in week 6 was real: keep its 126 crates."
println("Order file cleaned successfully!")
```

> **Note**
> This is the same file as in the lecture's game. In lecture 5, the cleaned orders come back in the bottling game, where you plan the brewery's bottling runs for these eight weeks.

Conclusion

Congratulations! You've successfully completed the tutorial on reading and writing external files in Julia. As a rule of thumb: use DelimitedFiles for quick numeric matrices, and CSV.jl together with DataFrames for real tabular data. In this course, we will mostly use the latter. Continue to the next file to learn more.

Solutions

You will likely find solutions to most exercises online. However, I strongly encourage you to work on these exercises independently without searching explicitly for the exact answers to the exercises. Understanding someone else's solution is very different from developing your own. Use the lecture notes and try to solve the exercises on your own. This approach will significantly enhance your learning and problem-solving skills.

Remember, the goal is not just to complete the exercises, but to understand the concepts and improve your programming abilities. If you encounter difficulties, review the lecture materials, experiment with different approaches, and don't hesitate to ask for clarification during class discussions.

Bibliography